

scatter/binomial

An AgentMPI collective scaling analysis

Abstract

The binomial scatter distributes work through a tree instead of from a single point. The root holds the contiguous span of virtual ranks $[0, 2^{\lceil \log_2 p \rceil})$ and hands the upper half of its span to a child, then the upper half of what remains to the next, and so on; each child repeats the split on the span it received. The root therefore issues $\lceil \log_2 p \rceil$ messages rather than $p - 1$, and the sweep confirms it: at $p = 32$ exactly 5 of the 31 sends came from rank 0, against 31 of 31 under the linear algorithm. Message count is $p - 1$ at all twenty-one measured sizes and the round count matches $\lceil \log_2 p \rceil$ everywhere, stepping only at 2, 4, 8, 16 and 32. The trade is volume: interior ranks forward blocks belonging to their subtree, so total traffic rises from $(p - 1)n$ to $np \lceil \log_2 p \rceil / 2$, 80 units against 31 at $p = 32$. The family view turns up a real inaccuracy in that volume term — it is exact only at powers of two and over-predicts elsewhere by up to 50 %, because the subtrees are truncated by p , while the implementation's self-report equals the exact truncated count at all thirteen distinct sizes. The property that makes the volume penalty acceptable is that the blocks passing through interior ranks are handles, not bodies: every receive in this sweep is logged `admitted: false` with `admitted_tokens: 0`, so no interior agent reads work assigned to somebody else.

1 What this family is

The `scatter` collective under the `binomial` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [\[ok\]](#) All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [\[note\]](#) Wall times span 0.003–0.125 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

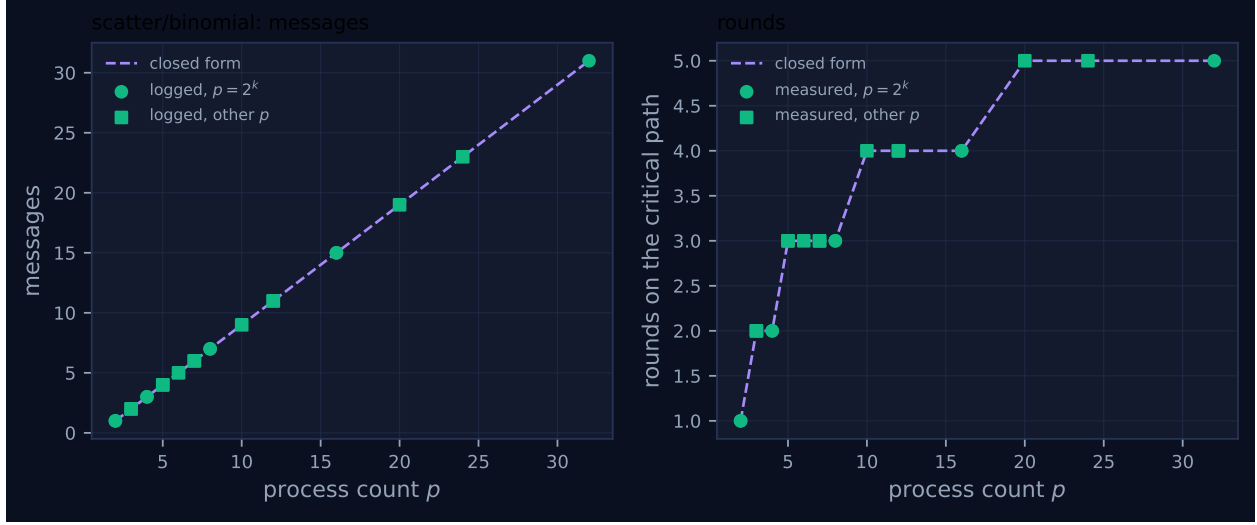


Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective’s own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	mb-free	1	1	1	1	1	0.003	✓	✓
2	mb-smoke	1	1	1	1	1	0.012	✓	✓
3	mb-free	2	2	2	2	2	0.012	✓	✓
3	mb-smoke	2	2	2	2	2	0.014	✓	✓
4	mb-free	3	3	3	2	2	0.027	✓	✓
4	mb-smoke	3	3	3	2	2	0.014	✓	✓
5	mb-free	4	4	4	3	3	0.017	✓	✓
5	mb-smoke	4	4	4	3	3	0.028	✓	✓
6	mb-free	5	5	5	3	3	0.036	✓	✓
7	mb-free	6	6	6	3	3	0.030	✓	✓
7	mb-smoke	6	6	6	3	3	0.031	✓	✓
8	mb-free	7	7	7	3	3	0.030	✓	✓
8	mb-smoke	7	7	7	3	3	0.028	✓	✓
10	mb-free	9	9	9	4	4	0.033	✓	✓
12	mb-free	11	11	11	4	4	0.037	✓	✓
12	mb-smoke	11	11	11	4	4	0.102	✓	✓
16	mb-free	15	15	15	4	4	0.054	✓	✓
16	mb-smoke	15	15	15	4	4	0.051	✓	✓
20	mb-free	19	19	19	5	5	0.111	✓	✓
24	mb-free	23	23	23	5	5	0.063	✓	✓
32	mb-free	31	31	31	5	5	0.125	✓	✓

4 How the algorithm scales

The tree is the same binomial tree the broadcast uses, with the payload split at each level rather than copied. A rank’s virtual rank is $vr = (r - \text{root}) \bmod p$; its parent clears its lowest set bit; its children are the nodes reachable by setting a strictly lower bit. The root starts with $\text{span} = 2^{\lceil \log_2 p \rceil}$

and a block indexed by virtual rank; each rank then walks $\text{mask} = \text{span}/2$ downwards, and for each existing child $vr + \text{mask}$ sends the block slice $[\text{child} - \text{base}, +\text{mask})$ and truncates its own block to what remains. What is left at the end, `block[0]`, is the rank's own unit.

Three costs follow. Every rank receives exactly once and only the root receives nothing, so the message count is $p - 1$, identical to the linear scatter. The critical path is the tree height, $\lceil \log_2 p \rceil$, because a rank cannot forward a slice it has not yet received. And the volume is not $(p - 1)n$: a rank forwarding at mask m ships m units, so summing over levels gives $np \lceil \log_2 p \rceil / 2$ — each of the $\lceil \log_2 p \rceil$ levels moving about $np/2$. That is `FORMULAS["scatter", "binomial"]`, and the docstring states the trade in as many words: “the total volume forwarded is $n \cdot p \cdot \log(p)/2$ rather than the $n \cdot (p-1)$ of the linear algorithm — the price of the logarithmic depth”.

Both checked terms agree at every size. The message panel of Figure 1 is the line $p - 1$ across all twenty-one rows; the round panel is the staircase $\lceil \log_2 p \rceil$, holding at 3 across $p = 5, 6, 7, 8$, at 4 across $p = 10, 12, 16$ and at 5 across $p = 20, 24, 32$. The steps land at the powers of two because the loop bound is derived from `_ilog2_ceil(p)`, so the height is $\lceil \log_2 p \rceil$ by construction rather than by accident.

What the panels cannot show is the redistribution of work away from the root, which is the whole purpose of the algorithm, and the log shows it directly. At $p = 32$ the 31 sends are distributed across exactly 16 ranks: rank 0 sends 5, rank 16 sends 4, ranks 8 and 24 send 3 each, ranks 4, 12, 20 and 28 send 2 each, and the eight even ranks 2, 6, 10, ..., 30 send 1 each, while the sixteen odd ranks send nothing and every rank from 1 to 31 receives exactly once. That is the binomial tree read straight off the message log — a rank's fan-out is the number of trailing zeros in its virtual rank — and it is the exact complement of the linear scatter, where all 31 sends came from rank 0. The viewer screenshot for `mb-free_coll-scatter-binomial-32` shows the same distribution as lane structure: no lane dominates, activity begins in one lane and spreads down the tree over the 0.12s span, and the deepest ranks receive last.

Aggregate receive-blocking at $p = 32$ is 0.46 rank-seconds against 0.32 for the linear scatter — higher, not lower, because interior ranks now wait for their parents before they can forward. That is the honest form of the trade and reporting it as a reduction would be wrong. What the tree reduces is the load on the one rank that cannot be replicated.

5 Powers of two and everything else

This is bit arithmetic, so this is where a defect would live. Two questions have to be separated: does the implementation behave, and does the closed form describe it?

The implementation behaves. At a non-power-of-two size the root's span, $2^{\lceil \log_2 p \rceil}$, is larger than p — 32 when $p = 20$ — so the block slices near the top of the range extend past the end of the payload list. The code handles this with two guards: `if child_v < p` suppresses sends to children that do not exist, and `hi = min(lo + mask, len(block))` truncates a slice at the end of the block it is cutting from. Together they mean an out-of-range subtree is never addressed and a partial subtree receives only its real members. The measurements confirm the result at all twenty-one sizes: 2, 4, 5, 6, 9, 11, 19 and 23 messages at $p = 3, 5, 6, 7, 10, 12, 20, 24$, with round counts 2, 3, 3, 3, 4, 4, 5 and 5 matching $\lceil \log_2 p \rceil$ throughout, no red crosses, and both campaigns agreeing at the eight duplicated sizes.

The closed form does not describe it exactly. The volume term assumes complete subtrees; the true traffic is the sum of *truncated* slice sizes, which equals $p \lceil \log_2 p \rceil / 2$ only when p is a power of

two. Comparing the exact count, the closed form, and the volume each run’s `coll.scatter` records reported:

p	reported (units)	exact	closed form	ratio
2	1	1	1.0	1.000
3	2	2	3.0	0.667
4	4	4	4.0	1.000
5	5	5	7.5	0.667
6	7	7	9.0	0.778
7	9	9	10.5	0.857
8	12	12	12.0	1.000
10	15	15	20.0	0.750
12	20	20	24.0	0.833
16	32	32	32.0	1.000
20	40	40	50.0	0.800
24	52	52	60.0	0.867
32	80	80	80.0	1.000

The reported volume equals the exact truncated count at every one of the thirteen sizes and equals the closed form at exactly the five powers of two; elsewhere the closed form over-predicts, worst at $p = 3$ and $p = 5$ where it is 1.5 times the truth. These are the same numbers `gather/binomial` produces, which is what duality requires and is a useful cross-check: two independently written branches, one splitting a block on the way down and one merging dictionaries on the way up, report identical volumes at all thirteen sizes.

The signature — exact at 2, 4, 8, 16, 32, a systematic excess everywhere else — is the one the repository already documents for recursive-doubling allreduce, and it is invisible in any single run for the same reason. Two things distinguish it and both should be said plainly. It sits in the *volume* term, which the generated table does not check — the table validates messages and rounds, and volume appears in neither column, which is why the family carries a [ok] rather than a warning. And it errs conservatively, over-stating cost rather than under-stating it, which is the opposite of the allreduce case where the count made the collective look cheaper than it was. It is still worth correcting, because `predict` derives `price_usd` from volume and a 50 % over-estimate of tokens at $p = 5$ is a 50 % over-estimate of the bill.

6 When to choose this algorithm

Against `linear` the comparison is clean because the message counts are identical at every size in both families, so nothing is traded in traffic and the whole difference is depth against volume. Depth: $\lceil \log_2 p \rceil$ against $p - 1$, or 5 against 31 at $p = 32$, and measured as root sends of 5 against 31. Volume: 80 units against 31, a factor of 2.6. `cost.predict` at $n = 4,096$ tokens per unit prices them at 2.58s against 15.97s of critical path and 327,680 against 126,976 tokens, and `cost.best_algorithm` accordingly answers `binomial` on time and `linear` on volume at every (p, n) I evaluated.

For agent ranks the decision goes to `binomial`, and the reason is the root. Work distribution under `linear` is a serial section of length $p - 1$ executed by one rank; if that rank is an agent it is $p - 1$ turns during which nothing else happens, and it is precisely the Amdahl term that agent harnesses cannot shrink — “the phases that must be done by one rank — plan, integrate, arbitrate”. Under `binomial` the root’s share is $\lceil \log_2 p \rceil$ and fifteen other ranks carry the rest. The volume penalty is only a real cost if the forwarded blocks move by value, and AgentMPI’s answer is that they need

not: the branch receives them with `admit=False`, so nothing an interior rank forwards is charged to its context account, and under rendezvous transport what it holds is a reference rather than a body. The docstring makes the point that this is what separates a scatter from “post the whole task list to the group chat”: under the group-chat pattern every rank pays context for the full p -way list, whereas here no rank pays context for anything, and interior ranks do not even see the content of what they forward.

The sweep shows the admission half of that mechanism working — all 31 receives at $p = 32$ logged `admitted: false` with `admitted_tokens: 0`, and all thirty-two ranks finalising with a context high-water of 0 against a 128,000-token budget — and cannot show the transport half, because `eager_limit=10**9` forced `Mode.AUTO` to eager for every message. For the transport half, the real eight-rank software run is the evidence, and it is unusually neat: within a single `blame-r1` scatter, rank 0 sent a 2,077-token block to rank 4 by `rendezvous` and 1,044- and 527-token blocks by `eager`, splitting at the 2,048-token default eager limit. Since the largest messages in a binomial scatter are the subtree blocks issued near the root, `Mode.AUTO` promotes exactly those to rendezvous — the ones carrying other ranks’ assignments — and leaves the leaf-bound single units eager. The transport policy and the tree structure line up without either being told about the other.

The one caveat is a sharp edge in that hard-coded `admit=False`. Because the literal replaces the caller’s argument, the `admit` keyword is silently ignored on this path while `linear` honours it, and `scatter()` defaults to “linear” if $p \leq 4$ else “binomial”. So `comm.scatter(units, admit=True)` places a rank’s work unit in its context at $p = 4$ and does not at $p = 5$. Nothing in this sweep exposes it, since every run names its algorithm and leaves `admit` at the default, but a harness that relies on `admit=True` will find it works at small p and stops working when the default flips.

7 Threats to this reading

No agents took part. No executor is recorded, the viewer reports zero agent calls, and the 0.003–0.125s wall times are SQLite writes and event-log appends. The blocking figures — 0.46 rank-seconds aggregate, root sends of 5 — are structural facts about the message pattern plus fabric timing under thread contention. The pattern is durable; the timing is not evidence about how long it takes to hand work to a model.

The work units are one token each, which makes the volume comparison small. At $p = 32$ the whole collective moves 80 payload tokens, so the 2.6-fold penalty over `linear` is 49 tokens. Every claim above about context and transport pressure is an argument from the closed form and from the real run, not a measurement here. The fabric’s own `msg.send` records report 593 tokens against the collective’s 80 at $p = 32$, because the fabric tokenises the whole `{"base": ..., "block": [...]}` envelope while the collective counts only the units; with one-token units the envelope dominates entirely, so that 7.4-fold ratio is a benchmark artefact that would shrink towards nothing with realistic payloads. I have used the payload figure throughout for comparability with the closed form.

The volume discrepancy is arithmetic I performed, not something the tooling flagged. The exact column is a truncated-slice sum evaluated over the measured grid; the reported column is `tokens_sent` summed over each run’s `coll.scatter` records. If the self-report were wrong in the same direction, the agreement would be two errors cancelling. Against that: the self-report sums `_tok()` over the items actually placed in each message, my figure is a pure combinatorial count, they agree to the unit at all thirteen sizes, and the independently written `gather/binomial` branch

produces the same thirteen numbers.

The round count is asserted, not observed. `tr.stats.rounds = _ilog2_ceil(p)` is computed from the same expression the closed form uses, so that column is a consistency check between two statements of one intent. The independent evidence for the tree height is the per-rank send distribution at $p = 32$ — fan-outs of 5, 4, 3, 3, 2, 2, 2, 2 and then eight ranks with 1, summing to 31 across 16 senders — which is consistent with a height-5 binomial tree and inconsistent with a flat fan-out.

The admit asymmetry is read from source, not measured. No run in this sweep passes `admit=True`. The claim rests on the literal `False` in this branch against the pass-through in `linear`, which is a short reading but not an observation; a test at $p = 4$ and $p = 5$ would settle it.